

Project - High Level Design on CI/CD for Go (Golang) Logistics App

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	JIYANSHI KESHRI	EN22CS303026
2	MANASVI JAIN	EN22CS301580
3	KRATIKA TOMAR	EN22CS301516
4	MADHURI PRAJAPATI	EN22CS301562
5	NIKHIL KUMAR SINGH	EN22CS301648
6	PRIYANKA KUMARI	EN22CS301758

Group Name: Placed Btech Group 05

Project Number: DO-05

Industry Mentor Name:

University Mentor Name:

Academic Year: 2026

1. Introduction

This document presents the High-Level Design (HLD) of the CI/CD Pipeline for Go Logistics Application. The objective of this project is to design a scalable and automated software delivery system using modern DevOps practices.

The system integrates version control, automated testing, containerization, and deployment pipelines to ensure efficient and reliable application delivery.

1.1 Scope of the document

This document covers:

- Overall system architecture
- CI/CD pipeline design
- Application deployment workflow
- Data handling strategy
- Security and performance considerations

This document does not include low-level implementation details.

1.2 Intended Audience

- DevOps Engineers
- Software Developers
- System Architects
- Project Mentors
- Stakeholders

1.3 System overview

The system is a Go-based Logistics Application integrated with a CI/CD pipeline.

Key features:

- Source code management using Git
- Automated build and testing
- Docker-based containerization
- CI/CD pipeline using GitHub Actions or Jenkins
- Deployment using container runtime

2 System Design

The system consists of:

- Source Code Repository
- CI/CD Pipeline
- Docker Containerization
- Deployment Environment

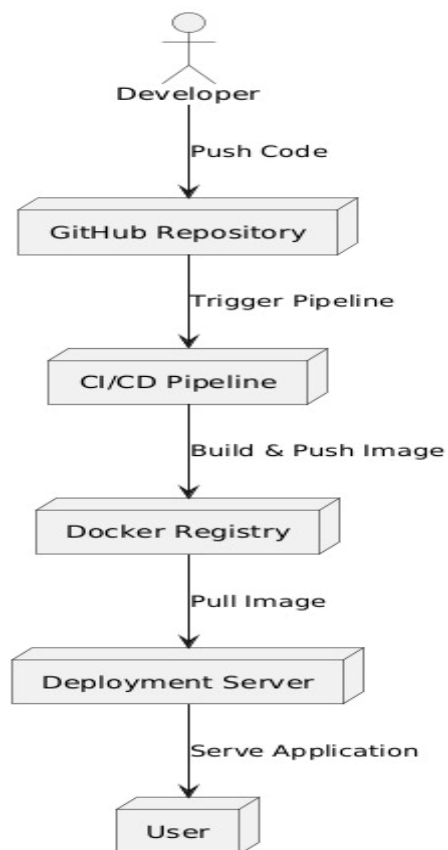


Fig.1 High level Architecture

2.1 Application Design

Application is a simple HTTP server handling logistics requests.

Main Components:

- HTTP Server
- Request Handler
- Business Logic Layer

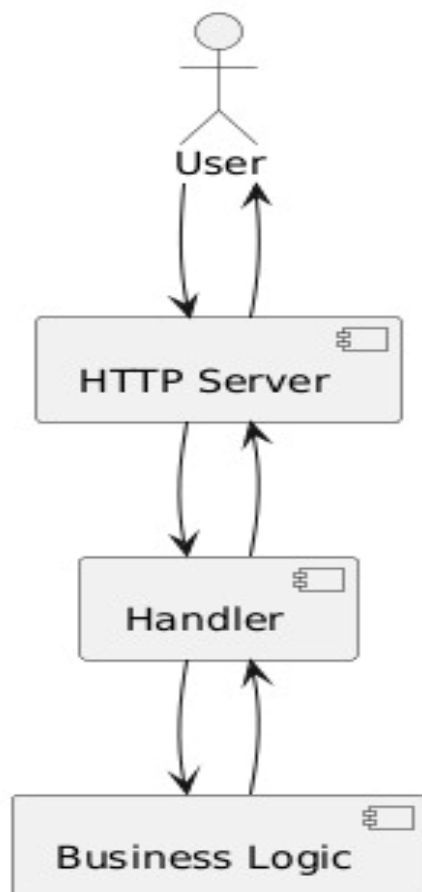


Fig.2 UML - Application Design

2.2 Process Flow

- Developer commits code to repository
- CI/CD pipeline triggers automatically
- Code is tested and built
- Docker image is created
- Image is pushed to registry
- Deployment server pulls image
- Application is deployed

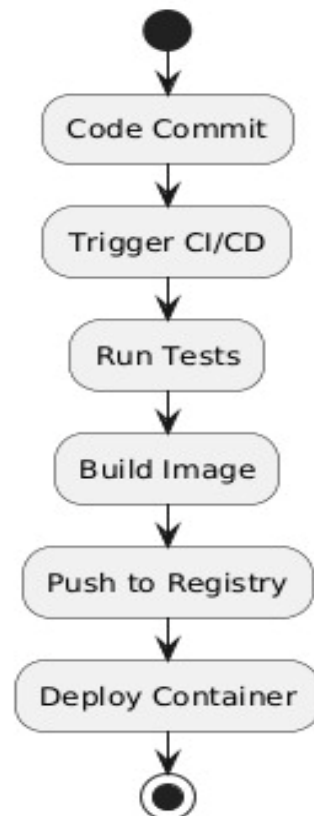


Fig.3 UML – Process Flow

2.3 Information Flow.

Information flows across multiple stages:

Code → GitHub
GitHub → CI/CD pipeline
Pipeline → Docker Registry
Registry → Deployment Server
Server → End User

2.4 Components Design

Main components:

- Git Repository
- CI/CD Tool (GitHub Actions / Jenkins)
- Docker Engine
- Container Runtime
- Deployment Server

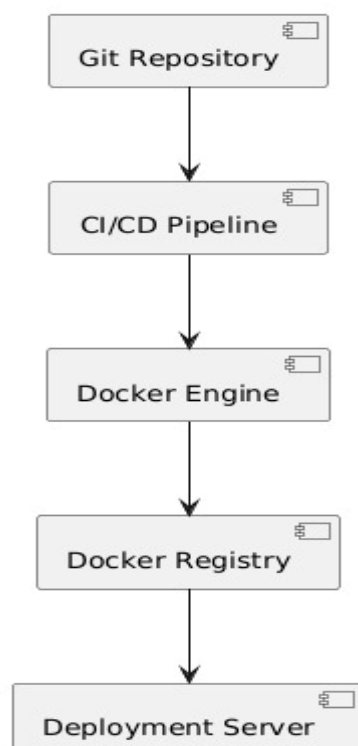


Fig.4 UML Component Diagram

2.5 Key Design Considerations

- Automation of build and deployment
- Scalability of application
- Fault tolerance
- Portability using containers
- Maintainability of pipeline

2.6 API Catalogue

Endpoint	Method	Description
/	GET	Check application status
/orders	GET	Fetch logistics orders
/track	POST	Track shipment

Data Design

The system manages lightweight data, including:

- Order details
- Tracking information
- Request logs

All data is structured and exchanged in JSON format, ensuring easy integration and scalability.

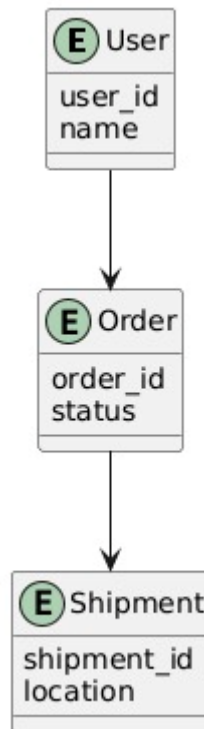


Fig.5 Data Model

3 Reference

Docker Official Docs

Go Official Documentation

DevOps CI/CD Best Practices